Trader Records API - Walkthrough and Rehearsal Guide

3-5 minute backend demonstration, rehearsal checklist, and access verification

Trader Records API

Document type	Delivery walkthrough
Version	1.0
Evidence cut-off	5 November 2026
Code baseline	chore/documentation-and-delivery (final delivery branch)
Prepared for	Recruitment submission
Repository	C:\Users\OYALE\Desktop\Project\TraderRecords

1. Scope and honest framing

This is a backend-only demonstration. The API supports retry-safe synchronisation, but the repository does not contain an implemented offline mobile client, background retry scheduler, chat frontend, or WhatsApp integration. Say that plainly if asked.

The timed narration is 3-5 minutes. Preparation, database reset, and reviewer-access checks happen outside the timer.

Table 1. What the demonstration proves and does not prove

Proves	Does not prove
A trader can register, log in, and receive a 60-minute JWT.	An offline mobile client exists or has been recorded.
A batch can accept valid items and reject invalid items without rolling back the batch.	The server can verify self-reported income or make a lending decision.
An identical retry is idempotent and a changed payload is a conflict.	A production deployment, load test, or disaster-recovery exercise exists.
Lender access is checked against the current persisted consent row.	Consent history is a complete audit trail.

2. Preparation and reset outside the timed narration

2.1 Prerequisites

- .NET SDK `9.0.316` pinned by `global.json`.
- ASP.NET Core Runtime `9.0.21` or a later compatible 9.0.x patch.
- A JWT signing key of at least 32 bytes in user-secrets or `Jwt__SigningKey`.
- The local `dotnet-ef` tool `9.0.21` from `.config/dotnet-tools.json`.

2.2 Reset the local demo database

Stop the API before deleting the database. This erases local demo records. There is no public reset endpoint.

```
Remove-Item -LiteralPath "TraderRecords.Api/App_Data/traderrecords.db",
    "TraderRecords.Api/App_Data/traderrecords.db-wal",
    "TraderRecords.Api/App_Data/traderrecords.db-shm" -ErrorAction SilentlyContinue
```

2.3 Start the API and open Scalar

```
dotnet tool restore
dotnet restore --locked-mode
dotnet build
$key = "<at-least-32-random-bytes>"
dotnet user-secrets set "Jwt:SigningKey" $key --project TraderRecords.Api
dotnet run --project TraderRecords.Api --launch-profile http
```

Open `http://localhost:5080/scalar/`. Confirm the bearer field, the endpoint list, and the role descriptions are visible before recording.

2.4 Demo identities

Table 2. Deterministic demo identities

Item	Value
Trader One login	`trader1@example.test` / `Trader1!Demo2026`
Trader Two login	`trader2@example.test` / `Trader2!Demo2026`
Lender login	`lender@example.test` / `Lender!Demo2026`
Trader One business ID	`aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa`
Trader Two business ID	`bbbbbbbb-bbbb-4bbb-8bbb-bbbbbbbbbbbb`
Seeded lender ID	`33333333-3333-4333-8333-333333333333`

2.5 Fresh client IDs and valid timestamps

Use fresh GUIDs for new sync items. Reuse the same `clientId` only when demonstrating an identical retry or a changed-payload conflict. Use a past timestamp

within the last 365 days. If the recording is made later, replace the example dates with the previous day.

```
# Fresh IDs for a new demonstration
11111111-1111-4111-8111-111111111111
22222222-2222-4222-8222-222222222222
44444444-4444-4444-8444-444444444444

# Valid past timestamp with an explicit offset
2026-11-04T09:00:00+01:00
```

2.6 Reviewer-access check

1. Open the repository URL in a signed-out or private browser window.

2. Open the published walkthrough video URL in the same private window when it exists.

3. Confirm the repository and video are accessible without your account session.

4. No video URL is committed yet. Do not present a placeholder as a live link.

Repository URL: `https://github.com/agbojisimon/TradersRecord`

3. Timed spoken script

0:00-0:30 - Problem and scope

Say:

> Many small traders keep sales and expenses in memory, paper, or chat messages. That makes it hard to show a reliable cash-flow history. This API lets a trader sync records from a client, see an eight-week summary, and grant a lender temporary access to that summary. The records are self-reported, so the indicator measures record consistency, not creditworthiness.

Keep the framing honest: this is a backend API, not an offline mobile product.

0:30-1:00 - Login

Table 3. Scalar action: trader login

Scalar field	Value
Endpoint	`POST /api/auth/login`
Account/token	`trader1@example.test` / `Trader1!Demo2026`
Request	JSON email and password; see the example below.
Expected response	`200` with `userId`, `businessId`, `role=Trader`, `accessToken`, `tokenType=Bearer`, and `expiresAtUtc`.

```
{
  "email": "trader1@example.test",
  "password": "Trader1!Demo2026"
}
```

Copy the returned `accessToken` into Scalar's bearer authorization field. Do not show the full token in the recording.

Say:

> The token is a 60-minute JWT. Ownership is resolved from the subject claim and the database, not from a business ID supplied by the caller. The login endpoint is in `AuthController.Login`; the credential check is in `AuthService.LoginAsync`; token creation is in `JwtTokenService.Create`; and JWT validation is configured in `DependencyInjection.cs`.

1:00-2:00 - Batch sync, identical retry, and changed-payload conflict

Table 4. Scalar action: first sync batch

Scalar field	Value
Endpoint	`POST /api/transactions/sync`
Account/token	Trader One bearer token from the login step.
Request	Three items: valid sale, invalid amount, valid expense; see the example below.
Expected response	`200` with `receivedCount=3`, `acceptedCount=2`, `rejectedCount=1`, and `INVALID_AMOUNT` on index 1.

```
{
  "items": [
    {
      "clientId": "11111111-1111-4111-8111-111111111111",
      "type": "Sale",
      "amountMinor": 4500000,
      "note": "Sold rice",
      "occurredAt": "2026-11-04T09:00:00+01:00"
    },
    {
      "clientId": "22222222-2222-4222-8222-222222222222",
      "type": "Sale",
      "amountMinor": -100,
      "note": "Invalid amount",
      "occurredAt": "2026-11-04T09:05:00+01:00"
    },
    {
      "clientId": "44444444-4444-4444-8444-444444444444",
      "type": "Expense",
      "amountMinor": 1200000,
      "note": "Stock purchase",
      "occurredAt": "2026-11-04T09:10:00+01:00"
    }
  ]
}
```

Point out that the response preserves input order. Index 1 is rejected; indexes 0 and 2 are accepted. One bad item does not discard the valid items.

Table 5. Scalar action: identical retry

Scalar field	Value
Endpoint	`POST /api/transactions/sync`
Account/token	Same Trader One bearer token.
Request	Send only the first sale again with the same `clientId`, amount, note, and timestamp.
Expected response	`200` with `duplicateCount=1`, `acceptedCount=0`, and the original transaction ID. No second row is created.

```
{
  "items": [
    {
      "clientId": "11111111-1111-4111-8111-111111111111",
      "type": "Sale",
      "amountMinor": 4500000,
      "note": "Sold rice",
      "occurredAt": "2026-11-04T09:00:00+01:00"
    }
  ]
}
```

Table 6. Scalar action: changed payload for the same client ID

Scalar field	Value
Endpoint	`POST /api/transactions/sync`
Account/token	Same Trader One bearer token.
Request	Reuse the same `clientId` but change `amountMinor` to `4600000`.
Expected response	`200` with `rejectedCount=1` and `CLIENT_ID_CONFLICT`. The original transaction is unchanged.

```
{
  "items": [
    {
      "clientId": "11111111-1111-4111-8111-111111111111",
      "type": "Sale",
      "amountMinor": 4600000,
      "note": "Sold rice",
      "occurredAt": "2026-11-04T09:00:00+01:00"
    }
  ]
}
```

Say:

> The client-generated ID is the idempotency key inside the trader's business. The unique database index on `(BusinessId, ClientId)` is the final guard against concurrent retries. An identical retry returns the original transaction ID. A changed payload is an item-level conflict, not a request-wide 409 and not an update. The endpoint is `TransactionsController.Sync`; the comparison and insert logic is in `TransactionSyncService.SyncAsync`; offset parsing is in `RequiredOffsetDateTimeOffsetConverter`.

> The batch is deliberately partial-success. Each readable item is validated independently, so a bad amount or note does not roll back valid entries. Envelope failures such as zero items, more than 100 items, malformed JSON, or an oversized body are request-wide failures.

2:00-2:40 - Trader summary

Table 7. Scalar action: trader summary

Scalar field	Value
Endpoint	`GET /api/businesses/me/summary`
Account/token	Trader One bearer token.
Request	No body. The business is resolved from the token subject.
Expected response	`200` with UTC window boundaries, exactly eight weekly buckets, integer-kobo totals, and the consistency indicator.

Point out the eight buckets, including empty weeks; integer kobo totals; `recordedDays`, `activeWeeks`, and the score; and the disclaimer that the score measures recording coverage, not verified income.

Say:

> Weeks start Monday at 00:00 Africa/Lagos. The current week is partial. The score is deterministic: 50 points for active days, 30 for active weeks, and 20 for history span, clamped to 0-100. The endpoint is `BusinessesController.GetSummary`; the window and totals are in `SummaryService.GetSummaryAsync`; the formula is in `ConsistencyIndicatorCalculator.Calculate`.

2:40-3:40 - Consent grant, lender access, and revocation

Table 8. Scalar action: lender summary before consent

Scalar field	Value
Endpoint	`GET /api/lender/businesses/{businessId}/summary`
Account/token	Lender bearer token: `lender@example.test` / `Lender!Demo2026`.
Request	Business ID `aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa`.
Expected response	`403 CONSENT_REQUIRED`.

Table 9. Scalar action: grant consent

Scalar field	Value
Endpoint	`PUT /api/businesses/me/consents/{lenderId}`
Account/token	Trader One bearer token.
Request	Lender ID `33333333-3333-4333-8333-333333333333`.
Expected response	`200` with `isActive=true`.

Table 10. Scalar action: lender summary after grant

Scalar field	Value
Endpoint	`GET /api/lender/businesses/{businessId}/summary`

Scalar field	Value
Account/token	Lender bearer token.
Request	Same business ID.
Expected response	`200` with the same eight-week summary contract.

Table 11. Scalar action: revoke consent

Scalar field	Value
Endpoint	`DELETE /api/businesses/me/consents/{lenderId}`
Account/token	Trader One bearer token.
Request	Same lender ID.
Expected response	`204 No Content`.

Table 12. Scalar action: lender summary after revocation

Scalar field	Value
Endpoint	`GET /api/lender/businesses/{businessId}/summary`
Account/token	Lender bearer token.
Request	Same business ID.
Expected response	`403 CONSENT_REQUIRED`.

Say:

> Consent is checked against the current persisted row on every lender summary request. Revocation therefore blocks the lender's next access check. A request already authorised before revocation may finish, but later requests are denied. Another lender's consent never grants access. Consent covers summary data only, not raw transaction notes or credentials. The endpoints are in `ConsentsController` and `LenderBusinessesController`; the rules are in `ConsentService`.

3:40-4:30 - Engineering close

Say:

> The API uses ASP.NET Core 9, EF Core SQLite, migrations, an idempotent development seeder, and one ProblemDetails error contract. OpenAPI and Scalar document bearer auth and role restrictions; the exported snapshot is `docs/openapi.json`. The delivery branch has 18 passing tests covering sync, summary, consent, roles, ownership, and concurrency. A separate authentication test branch adds focused login success and invalid-credential cases.

> The main limitation is scope: there is no offline client, background retry scheduler, refresh token, password recovery, or production deployment. The summary is based on self-reported records and is not a credit decision. Retry-safe backend sync support does not demonstrate an implemented offline mobile client.

4. Rehearsal checklist

- Reset the demo database and restart the API.
- Confirm `/api/health`, `/openapi/v1.json`, and `/scalar/` return successfully.
- Log in as Trader One and paste the token into Scalar.
- Send the three-item batch and confirm partial success.
- Repeat the valid sale with the identical `clientId` and confirm `Duplicate`.
- Change only the amount on that `clientId` and confirm `CLIENT_ID_CONFLICT`.
- Call the trader summary and point out eight buckets and integer kobo.
- Call the lender summary before consent and confirm `403 CONSENT_REQUIRED`.
- Grant consent, call the lender summary, revoke consent, and call it again.
- Confirm no signing key, full token, or real financial data is visible.
- Check the repository and video access in a signed-out/private browser window.

5. Common demonstration mistakes

- Reusing an old `clientId` for a genuinely new transaction, which produces a duplicate or conflict.
- Changing the timestamp or note during an identical retry, which turns the retry into a conflict.
- Using a future timestamp or one older than 365 days.
- Using the lender token on a trader-only route, or forgetting to paste the trader token.
- Granting consent with the wrong lender ID or the wrong business ID.
- Expecting a cached Scalar response to update after revocation; execute the request again.
- Showing the JWT signing key, a full bearer token, or real financial data.
- Claiming the indicator is a credit score or that an offline mobile client exists.
- Recording against a stale local database that was not reset.

6. Fallback and recovery

- `INVALID_OCCURRED_AT`: replace the example timestamp with the previous day and keep an explicit offset.
- `401 UNAUTHORIZED`: log in again and paste the new token into Scalar.
- `409 EMAIL_IN_USE`: use the seeded login instead of registering a duplicate email.
- `DATABASE_BUSY`: stop the API, reset the demo database, and restart.
- Consent already active: revoke it, then grant it again for a clean demonstration.
- Scalar does not load: confirm the API is listening on `http://localhost:5080` and open `/scalar/`.

7. File and method reference

Table 13. Implementation references used in the walkthrough

Behaviour	File	Method or type
Login endpoint	`API/Controllers/AuthController.cs`	`Login`
Login and password check	`Application/Services/AuthService.cs`	`LoginAsync`
JWT creation	`Infrastructure/Authentication/JwtTokenService.cs`	`Create`
JWT validation	`Infrastructure/DependencyInjection.cs`	JWT bearer configuration
Sync endpoint	`API/Controllers/TransactionsController.cs`	`Sync`
Sync validation and idempotency	`Application/Services/TransactionSyncService.cs`	`SyncAsync`
Offset timestamp parsing	`API/Json/RequiredOffsetDateTimeOffsetConverter.cs`	`Read`
Summary endpoint	`API/Controllers/BusinessesController.cs`	`GetSummary`
Summary window and totals	`Application/Services/SummaryService.cs`	`GetSummaryAsync`
Indicator formula	`Application/Services/ConsistencyIndicatorCalculator.cs`	`Calculate`
Consent grant and revoke	`API/Controllers/ConsentsController.cs`	`Grant`, `Revoke`
Lender summary endpoint	`API/Controllers/LenderBusinessesController.cs`	`GetSummary`
Consent rules	`Application/Services/ConsentService.cs`	grant, revoke, active-consent checks
ProblemDetails mapping	`API/ErrorHandling/ResultExtensions.cs`	`ToActionResult`
Unexpected exception handling	`API/ErrorHandling/GlobalExceptionHandler.cs`	`TryHandleAsync`

8. Submission access verification

1. Open `https://github.com/agbojisimon/TradersRecord` in a private or signed-out browser window.

2. Confirm the repository is public and the README renders without authentication.

3. Open `docs/TraderRecords_Walkthrough_and_Rehearsal_Guide.pdf` and the other delivery documents from the repository.

4. Open the published video URL in the same private window when it exists.

5. Confirm the video plays without requesting account access.

6. Record the date, browser, and result of the access check.

The video URL is not yet published. No placeholder URL is presented as live; add the real URL only after recording and publishing.

9. Known limitations and honest claims

- Backend synchronisation support exists; an offline mobile client does not.
- No background retry scheduler, chat frontend, WhatsApp integration, or natural-language parsing.
- No refresh tokens, MFA, password reset, email verification, or account lockout.
- No production hosting, TLS termination, CORS policy, rate limiting, load test, or disaster recovery.
- SQLite is a single-writer demo database; production would need a server database.
- Consent is current-state, not an append-only history.
- The consistency indicator measures record coverage, not creditworthiness.
- The generated OpenAPI snapshot does not include explicit examples and does not document a 500 response.
- Focused authentication tests are on a separate test/authentication branch and PR.

10. Exact reset and run commands

```
# Reset local demo data
Remove-Item -LiteralPath "TraderRecords.Api/App_Data/traderrecords.db",
    "TraderRecords.Api/App_Data/traderrecords.db-wal",
    "TraderRecords.Api/App_Data/traderrecords.db-shm" -ErrorAction SilentlyContinue

# Restore, build, test
dotnet tool restore
dotnet restore --locked-mode
dotnet build --no-restore
dotnet test --no-build --no-restore

# Configure the signing key and run
dotnet user-secrets init --project TraderRecords.Api
$key = "<at-least-32-random-bytes>"
dotnet user-secrets set "Jwt:SigningKey" $key --project TraderRecords.Api
dotnet run --project TraderRecords.Api --launch-profile http
```